

Raksha

R.A.K.S.H.A - Real-time AI Knowledge Security & Hazard Assessment

Operating Enterprise: YesPanchi Group of Companies

Document Title: Enterprise Technical Architecture, Port Specifications & Operations Manual

Document Code: RAKSHA-ENG-SPEC-2026-V2.4

Classification: Enterprise Confidential - Authorized Internal Engineering & DevOps

Current Baseline: Version 2.4.0 (Enterprise Release)

Publication Date: October 2026

Executive Platform Summary

RAKSHA (Real-time AI Knowledge Security & Hazard Assessment) is an enterprise cybersecurity and AI governance platform designed by YesPanchi Group of Companies. It establishes zero-trust data protection across enterprise Generative AI usage:

- **Centralized API Gateway:** High-performance proxy built on Go 1.26 fasthttp mediating all application requests to LLM providers (OpenAI, Anthropic, Bedrock, Ollama) with prompt guardrails, PII masking, rate-limiting, and cost controls.
- **Desktop Browser Guard:** Silent workstation agent and local MitM proxy built on Python 3.11 mitmproxy protecting employee browser sessions to public AI platforms (ChatGPT, Claude, Gemini) with real-time file upload blocking and DLP.

Metadata Field	Platform Specification Details
Platform Name	Raksha (Real-time AI Knowledge Security & Hazard Assessment)
Enterprise Parent	YesPanchi Group of Companies
Core Architecture	Dual-Plane: Go API Gateway (Central) + Python MitM Agent (Endpoint)
Primary Database	PostgreSQL 16 Enterprise (SCRAM-SHA-256 TLS Authentication)
Orchestration	Multi-container Docker Compose on Alpine Linux 3.23.4 Hardened Base
Target Workstations	Microsoft Windows 10/11, Apple macOS Sonoma/Sequoia, Linux Servers

Table of Contents

Section No.	Chapter Title	Topic Scope & Subject Matter	Target Page
Section 1	System Architecture & Dual-Plane Overview	Centralized API Gateway, Endpoint Browser Guard, and Data Flow	Page 3
Section 2	Languages, Frameworks & Version Matrix	Pinned versions for Go, React, TypeScript, Python, Alpine, and PostgreSQL	Page 4
Section 3	Network Architecture & Port Matrix	Complete port reference (6000, 3100, 18182, 18103, 18195, 5432, 11434, 443)	Page 5
Section 4	Docker Installation & Command Handbook	1-line installation scripts and complete container operations CLI	Page 7
Section 5	Cryptographic Secrets & Certificates Handbook	OpenSSL generation for .env secrets, Certbot HTTPS, and Root CA setup	Page 8
Section 6	Production Deployment & Verification Matrix	Quick-start checklist, health endpoints, and troubleshooting matrix	Page 10

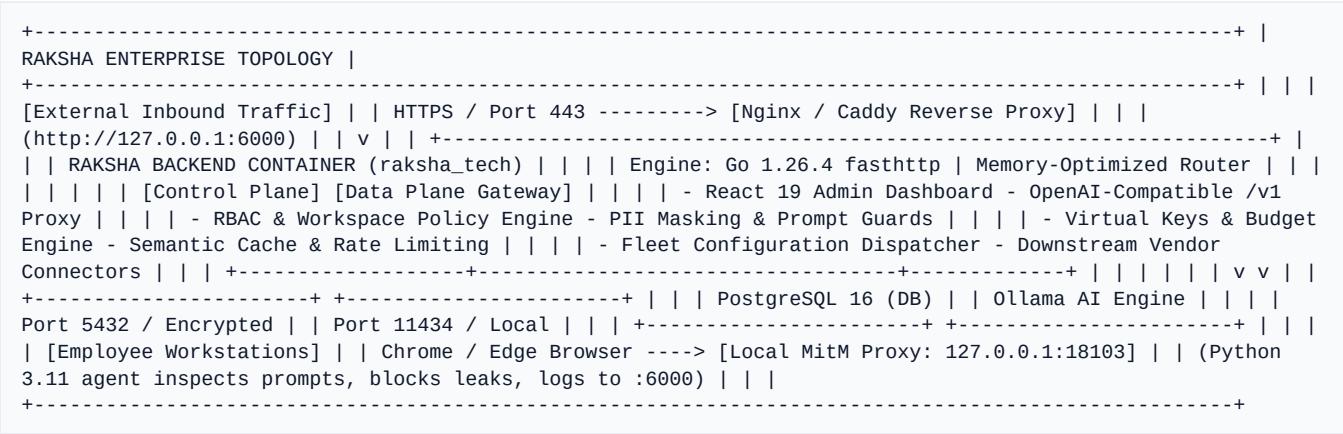
Standard Operating Guidelines

- 1. Deterministic Execution:** All terminal commands are standardized for bash, sh, and PowerShell execution.
- 2. Zero-Trust Security:** Default passwords, test tokens, and example encryption keys must be regenerated before production deployment.
- 3. Configuration Authority:** All runtime variables declared across this manual originate from the root **.env** file.

1. System Architecture & Dual-Plane Overview

Raksha operates a decoupled **dual-plane architecture** that protects both backend server integrations and employee desktop browser interactions through a unified management plane.

1.1 High-Level Architecture Flowchart



1.2 Dual-Plane Functional Responsibilities

1. Central Gateway Plane (Port 6000):

- High-throughput API mediation proxying internal microservices to ~28 cloud AI vendors (OpenAI, Anthropic, Bedrock, Gemini, Cohere, etc.).
- Applies pre-flight token bucket rate-limits, virtual key spend quotas, prompt injection guards, and asynchronous audit logging.

2. Endpoint Browser Guard Plane (Port 18103):

- Lightweight desktop service running silently on employee laptops.
- Intercepts web browser traffic to public Generative AI websites (ChatGPT, Claude, Gemini).
- Enforces real-time Data Loss Prevention (DLP): blocks unauthorized document uploads (PDF, DOCX, ZIP), redacts PII, and displays enterprise compliance banners.

2. Languages, Frameworks & Version Matrix

Every component of the Raksha platform is pinned to exact production releases to guarantee stability, security, and reproducible builds:

Component / Layer	Technology	Exact Version	Source File in Repo	Purpose & Function
Backend Core	Go (Golang)	1.26.4	go.work, core/go.mod	High-throughput gateway, proxying, and RBAC
HTTP Engine	valyala/fasthttp	v1.69.0	transports/raksha-http	Low-latency, zero-memory-allocation HTTP engine
Database ORM	GORM	v1.31.1	framework/configstore	Schema migrations & PostgreSQL driver
Embedded Database	mattn/go-sqlite3	v1.14.33	CGO Static Link	Fast local caching and offline fallback storage
CPU Optimizer	automaxprocs	v1.6.0	transports/raksha-http	Automatic container CPU quota thread tuning
Frontend Framework	React	19.2.3	ui/package.json	Modern component-driven administration dashboard
Frontend Language	TypeScript	5.9.2	ui/package.json	Strict type safety across the entire UI surface
UI Build System	Vite	8.0.16	ui/package.json	High-speed frontend bundler and dev server
UI Navigation	TanStack Router	1.168.10	ui/package.json	Type-safe client-side routing & page layouts
Styling Engine	Tailwind CSS	4.1.12	ui/package.json	Utility-first responsive design framework
Desktop Agent	Python	3.11+ / 3.12	apps/browser-guard	Workstation agent & local MitM interceptor
Interception Engine	mitmproxy	>= 10.0.0	apps/browser-guard	Real-time GenAI HTTP/HTTPS traffic analysis & DLP
Binary Compiler	PyInstaller	>= 6.0.0	apps/browser-guard	Compiles Python agent into standalone EXE / PKG
Windows Installer	Inno Setup	6.2+	apps/browser-guard	Enterprise Windows silent MSI/EXE installer
Node Build Runtime	Node.js Alpine	Node 25-alpine	Dockerfile.local	Docker multi-stage frontend compilation stage
Container Base OS	Alpine Linux	3.23.4	Dockerfile.local	Minimal, hardened container runtime environment
Primary Database	PostgreSQL	16.x	docker-compose.yml	Production database for config, keys, and audit logs
Local AI Engine	Ollama	v0.3+ / v0.5+	docker-compose.yml	Offline local LLM model execution (Port 11434)

3. Network Architecture & Port Matrix

Raksha requires specific network port assignments for inter-container communication, workstation proxying, and secure public ingress:

3.1 Network Port Allocation Table

Port Number	Protocol	Config Key	Scope / Binding	Service Name	Technical Function & Description
6000	TCP	APP_PORT	0.0.0.0:6000	raksha_tech	Primary Go backend. Serves API gateway, Web UI, and Browser Guard fleet reporting.
3100	TCP	UI_PORT	127.0.0.1:3100	Vite Dev Server	Local frontend development server (<code>npm run dev</code> in <code>ui/</code>). Inactive in production.
18182	TCP	PROXY_PORT	0.0.0.0:18182	Network Proxy	Optional shared MitM proxy container for centralized office PAC deployments.
18103	TCP	RAKSHA_PROXY_ADD R	127.0.0.1:18103	Laptop Guard Proxy	Local MitM proxy running on each employee laptop to inspect outbound GenAI traffic.
18195	TCP	PAC_HTTP_PORT	127.0.0.1:18195	Laptop PAC Server	Local HTTP server serving the dynamic Proxy Auto-Configuration (PAC) script to the OS.
18183	TCP	MITM_WEB_PORT	127.0.0.1:18183	Dev Web Inspector	Diagnostic web interface for inspecting mitmproxy flows during local development.
5432	TCP	DB_PORT	Internal VPC	PostgreSQL	Production relational database storing configuration, tenants, keys, and audit logs.
11434	TCP	OLLAMA_URL	Docker Bridge	Ollama AI Engine	Local AI inference engine serving offline open-source models via <code>1panel-network</code> .
80	TCP	System Port	0.0.0.0:80	Nginx / Caddy	Inbound public HTTP port for Certbot ACME domain validation and HTTPS redirect.

Port Number	Protocol	Config Key	Scope / Binding	Service Name	Technical Function & Description
443	TCP	System Port	0.0.0.0:443	Nginx / Caddy	Public secure HTTPS edge entry point terminating SSL/TLS certificates.

3.2 Network Perimeter & Firewall Security Matrix

+-----+					
PERIMETER FIREWALL RULES					
+-----+-----+-----+-----+					
Traffic Policy Port Allowed Source IP Security Policy & Purpose					
+-----+-----+-----+-----+					
PUBLIC INGRESS 443 0.0.0.0/0 (Internet) Main entry point terminating TLS for UI/API PUBLIC INGRESS					
80 0.0.0.0/0 (Internet) ACME HTTP-01 challenge & redirect to HTTPS INTERNAL PRIVATE 6000					
127.0.0.1 / Nginx Host Protect Go backend from direct public access INTERNAL PRIVATE 5432 App					
Containers / VPC Restrict PostgreSQL database to VPC only INTERNAL PRIVATE 11434 App Containers /					
VPC Restrict Ollama LLM to internal backend ENDPOINT LOOPBACK 18103 127.0.0.1 Only Workstation					
proxy bound strictly to local ENDPOINT LOOPBACK 18195 127.0.0.1 Only Workstation PAC server bound					
to local					
+-----+-----+-----+-----+					

4. Docker Installation & Command Handbook

Raksha is deployed and managed via Docker Compose. Below are one-line installation scripts and an exhaustive daily operations command handbook.

4.1 Download & Install Docker (One-Liner Commands)

- **Ubuntu / Debian Linux (Ubuntu 20.04, 22.04, 24.04 LTS):**

```
curl -fsSL https://get.docker.com | sh && sudo systemctl enable --now docker && sudo usermod -aG docker $USER
```

- **CentOS / RHEL 8 & 9 / Rocky Linux:**

```
sudo dnf install -y dnf-plugins-core && sudo dnf config-manager --add-repo
https://download.docker.com/linux/centos/docker-ce.repo && sudo dnf install -y docker-ce docker-ce-cli
containerd.io docker-compose-plugin && sudo systemctl enable --now docker
```

- **Microsoft Windows 10 / 11:**

1. Open PowerShell as Administrator and run: **ws1 --install**
2. Download Docker Desktop from: **<https://docs.docker.com/desktop/install/windows-install/>**
3. Ensure the "Use WSL 2 instead of Hyper-V" setting is enabled.

4.2 Docker Network Prerequisites (Run Once Before Starting)

```
# Create dedicated network for Ollama container integration docker network create lpanel-network # Create
primary application bridge network for Raksha docker network create raksha-network # Verify network creation
docker network ls
```

4.3 Complete Operations Command Handbook

```
# --- STARTUP & BUILD OPERATIONS --- # 1. Start primary Raksha backend in background (detached mode) docker
compose up -d # 2. Start primary backend AND the optional shared network proxy docker compose --profile
network-proxy up -d # 3. Rebuild and launch containers after any source code change docker compose up -d
--build # 4. Force clean rebuild without utilizing Docker cache docker compose build --no-cache && docker
compose up -d # --- MONITORING & LOG OPERATIONS --- # 5. Check status, healthchecks, and uptime of running
containers docker compose ps # 6. Follow real-time live logs of the primary Go backend container docker
compose logs -f --tail=100 raksha_tech # 7. Check container CPU percentage, RAM consumption, and network I/O
docker stats raksha_tech # --- IN-CONTAINER DEBUGGING OPERATIONS --- # 8. Open an interactive shell inside
the running Raksha backend container docker exec -it raksha_tech /bin/sh # 9. Verify internal health check
endpoint from inside the container docker exec -it raksha_tech wget -qO- http://127.0.0.1:6000/health # ---
STOP, RESTART & MAINTENANCE OPERATIONS --- # 10. Gracefully restart the backend container (e.g. after
modifying .env) docker compose restart raksha_tech # 11. Stop containers without deleting container instances
or networks docker compose stop # 12. Stop and remove containers, networks, and internal links docker compose
down # 13. Stop and remove containers + internal volumes (bind mounts preserved) docker compose down -v # 14.
Clean up stopped containers, unused networks, and dangling images docker system prune -f
```

5. Cryptographic Secrets & Certificates Handbook

All cryptographic keys, authentication tokens, and SSL certificates must be generated using cryptographically secure random sources prior to production launch.

5.1 Secrets for **.env** File

Environment Secret Key	Cryptographic Standard	Technical Purpose	Exact Generation Command
RAKSHA_GUARD_SECRET	256-bit Random Base64	Fleet Desktop Agent Mutual Auth Token	openssl rand -base64 32
RAKSHA_ENCRYPTION_KEY	256-bit Random Base64	AES-256 DB Credential Encryption at Rest	openssl rand -base64 32
PASSWORD_RESET_SECRET	384-bit Random Base64	User Password Recovery HMAC Token	openssl rand -base64 48
RAKSHA_METRICS_TOKEN	256-bit Random Hex	Prometheus Bearer Token for /metrics	openssl rand -hex 32
CLUSTER_REPLICATE_SECRET	384-bit Random Base64	Multi-Node Cluster KV State Sync Secret	openssl rand -base64 48

```
# Automated one-liner to generate and inject all secrets directly into .env: sed -i
"s|^RAKSHA_GUARD_SECRET=. *|RAKSHA_GUARD_SECRET=$(openssl rand -base64 32)|" .env sed -i
"s|^RAKSHA_ENCRYPTION_KEY=. *|RAKSHA_ENCRYPTION_KEY=$(openssl rand -base64 32)|" .env sed -i
"s|^PASSWORD_RESET_SECRET=. *|PASSWORD_RESET_SECRET=$(openssl rand -base64 48)|" .env sed -i
"s|^RAKSHA_METRICS_TOKEN=. *|RAKSHA_METRICS_TOKEN=$(openssl rand -hex 32)|" .env sed -i
"s|^CLUSTER_REPLICATE_SECRET=. *|CLUSTER_REPLICATE_SECRET=$(openssl rand -base64 48)|" .env
```

5.2 Public SSL/TLS Certificate Generation (Certbot / Let's Encrypt)

```
# 1. Install Certbot sudo apt update && sudo apt install -y certbot python3-certbot-nginx # 2. Acquire public
certificate in standalone mode sudo certbot certonly --standalone -d raksha.yourcompany.com --email
secops@yourcompany.com --agree-tos # 3. Setup automated bi-daily renewal cron job echo "0 3,15 * * * root
certbot renew --quiet --post-hook 'systemctl reload nginx'" | sudo tee -a /etc/crontab
```

5.3 Private Self-Signed TLS Certificate Generation (OpenSSL 4096-bit with SAN)

```
# Generate 4096-bit RSA certificate valid for 3 years (1095 days) openssl req -x509 -nodes -days 1095 -newkey
rsa:4096 \ -keyout raksha_server.key -out raksha_server.crt \ -subj
"/C=US/ST=State/L=City/O=YesPanchi/OU=IT/CN=raksha.yourcompany.com" \ -addext
"subjectAltName=DNS:raksha.yourcompany.com,DNS:localhost,IP:127.0.0.1"
```

5.4 Browser Guard Root CA Generation (for HTTPS Traffic Interception)

```
# 1. Generate Root CA Private Key (4096-bit RSA) openssl genrsa -out raksha-ca.key 4096 # 2. Generate Root CA
Certificate (valid for 10 years / 3650 days) openssl req -x509 -new -nodes -key raksha-ca.key -sha256 -days
3650 \ -out raksha-ca.crt -subj "/CN=Raksha Root CA - Enterprise AI Guard" # 3. Combine into single PEM for
mitmproxy cat raksha-ca.key raksha-ca.crt > mitmproxy-ca.pem
```

5.5 Deploying Root CA to Client OS Trust Stores

- **Windows Clients (Execute in CMD / PowerShell as Administrator):**

```
certutil -addstore -f "Root" "C:\path\to\raksha-ca.crt"
```

- **macOS Clients (Execute in Terminal with sudo):**

```
sudo security add-trusted-cert -d -r trustRoot -k /Library/Keychains/System.keychain /path/to/raksha-ca.crt
```

- **Linux Workstations (Ubuntu / Debian):**

```
sudo cp raksha-ca.crt /usr/local/share/ca-certificates/ && sudo update-ca-certificates
```

6. Production Deployment & Verification Matrix

6.1 Quick-Start 4-Step Deployment Checklist

+-----+									
PRODUCTION DEPLOYMENT CHECKLIST									
+-----+									
Step	Action Task	Terminal Command to Execute							
+-----+									
Copy Environment File	cp .env.example .env	2 Generate Secrets	Populate RAKSHA_GUARD_SECRET &	1					
RAKSHA_ENCRYPTION_KEY in .env	3 Create Docker Network	docker network create 1panel-network	4						
Compile & Start App	docker compose up -d --build								
+-----+									

6.2 Health Checks & Operational Verification

1. Verify Go Backend Healthcheck Endpoint curl -i http://localhost:6000/health # Expected: HTTP/1.1 200 OK {"status":"healthy"} # 2. Verify Container Process Status docker compose ps # Expected: raksha_tech (healthy) Up X minutes 0.0.0.0:6000->6000/tcp # 3. Test PostgreSQL Database Connectivity from Host nc -zv 127.0.0.1 5432 # Expected: Connection to 127.0.0.1 port 5432 [tcp/postgresql] succeeded! # 4. Test Local Ollama Model Server Connectivity curl -i http://127.0.0.1:11434/api/tags # Expected: HTTP/1.1 200 OK with JSON array of installed models

6.3 Operational Diagnostics & Resolution Matrix

Symptom / Observed Error	Root Cause Analysis	Corrective CLI Resolution
failed to connect to PostgreSQL on port 5432	Database container down or firewall blocking port	Run docker compose ps to inspect DB; check DB credentials in .env ; verify host port.
network 1panel-network not found	External network was not created prior to launch	Run docker network create 1panel-network , then rerun docker compose up -d .
Browser Guard: 401 Unauthorized on /api/*	Mismatched RAKSHA_GUARD_SECRET between client & backend	Run python apps/browser-guard/scripts/sync_config_from_env.py to synchronize keys.
Browser displays ERR_CERT_AUTHORITY_INVALID	Root CA certificate not installed in client trust store	Install raksha-ca.crt into client OS store (certutil on Windows, security on macOS).
Port 6000 already in use / bind: address already in use	Another process is occupying host port 6000	Run netstat -ano
UI displays blank screen or 404 on assets	Web UI static bundle missing from Go binary	Run npm run build:raksha in ui/ , then rebuild with docker compose up -d --build .

Platform Support & Maintenance Contacts

- **Lead Repository:** **d:\unifai_project**
- **Core Configuration:** **.env** and **configs/config.json**
- **Platform Architecture Team:** Raksha SecOps & Cloud Infrastructure Group, YesPanchi Group of Companies